

CHAPTER 01

01. JavaScript + TypeScript 기초



Node.js · Express · NestJS 강의 · 2026

서버를 만들기 전에, 서버를 만들 도구부터 손에 익혀야 한다. 이 과정의 모든 코드는 JavaScript 또는 TypeScript로 쓰인다. Express 라우트도, NestJS 컨트롤러도 결국은 변수·함수·객체·배열·비동기 처리의 조합이다. 이 장은 그 바탕을 다진다.

순서가 중요하다. **먼저 JavaScript를 끝낸 뒤 TypeScript로 넘어간다.** JS는 타입이 없는 언어라 자유롭지만, 그만큼 실수가 런타임에야 드러난다. JS를 충분히 다뤄 보면 "여기서 문자열이 들어오면 터지는데 컴파일러가 미리 막았으면" 하는 순간이 온다. 그 순간 TS가 무엇을 해결하는지 보인다. TS는 JS에 타입이라는 안전장치를 더하며, 두 언어는 같은 문법을 공유한다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- JavaScript 핵심: 변수·연산자·타입, 제어문, 함수와 콜백, 객체·클래스, 비동기(콜백·Promise·async/await), 구조 분해, 스프레드/레스트, 클로저, 배열 고차 함수
- TypeScript가 JS 위에 더하는 것: 기본 타입, 인터페이스와 타입 별칭, 제네릭, 접근 제어자가 있는 클래스, 유니온·타입 가드, 유틸리티 타입, 데코레이터
- 두 언어로 작은 실전 코드(HTTP 서버, REST 클라이언트, Todo 앱)를 읽고 구조를 파악하는 법

내용은 크게 두 묶음, JavaScript와 TypeScript로 나뉜다. 각 주제는 **독립 실행** 스크립트로 익히며, 한 번에 하나씩 돌려 출력을 확인한다. JS는 `node` 로, TS는 `tsx` 로 실행한다.

주제	핵심
출력과 문자열	출력·템플릿 문자열·연산자
변수와 타입 변환	변수, 증감/복합 할당, 명시적·자동 변환
타입 확인과 동등 비교	<code>typeof</code> , <code>=</code> vs <code>===</code> , <code>const</code> , 블록 스코프
조건문과 단축 평가	<code>if</code> / 삼항 / <code>switch</code> , 단축 평가, ??
배열과 반복문	배열, <code>while</code> / <code>for</code> / <code>for.of</code> , <code>break</code> / <code>continue</code>
함수와 콜백	함수 3종, 콜백, 타이머
객체와 클래스	객체·메서드· <code>this</code> , 클래스·상속·정적
에러 처리와 비동기	<code>try/catch/finally</code> , 콜백→ <code>Promise</code> → <code>async/await</code>
구조 분해	배열·객체 구조 분해, 기본값, 중첩
스프레드와 레스트	스프레드/레스트, 불변 업데이트
클로저와 스코프	스코프·호이스팅·TDZ, 클로저
배열 고차 함수	<code>filter</code> / <code>map</code> / <code>reduce</code> / <code>find</code> / <code>some</code> / <code>every</code> / <code>sort</code>
기본 타입	기본 타입, 튜플, <code>enum</code> , <code>any</code> / <code>void</code> / <code>never</code> , 유니온
인터페이스와 타입 별칭	인터페이스, 선택/읽기전용, 함수 타입, 타입 별칭, 확장
함수와 제네릭	선택/기본/나머지 매개변수, 오버로드, 제네릭
클래스와 접근 제어자	접근 제어자, 추상 클래스, <code>implements</code> , <code>getter/setter</code>
모듈과 네임스페이스	<code>export</code> / <code>import</code> , 네임스페이스
유니온과 판별 유니온	유니온, 판별 유니온(<code>kind</code> 로 분기)
타입 가드	타입 가드(<code>is</code>), 타입 단언, 타입 추론
유틸리티 타입	<code>Partial</code> / <code>Required</code> / <code>Readonly</code> / <code>Record</code> / <code>Pick</code> / <code>Omit</code>
데코레이터	클래스·메서드·프로퍼티 데코레이터(<code>reflect-metadata</code>)
타입과 함께 보는 비동기	<code>Promise</code> , <code>async/await</code> , <code>Promise.all</code> / <code>race</code> , 체이닝
Node 모듈과 타입	<code>fs</code> / <code>http</code> / <code>dotenv</code> 등 Node 모듈 + 타입

주제	핵심
실전 Todo 앱	타입을 엮은 작은 Todo 앱(싱글턴·제네릭·로거)

핵심 개념 — 타입은 "실행 전에 잡는 안전장치"다

JavaScript는 변수에 타입을 붙이지 않는다. `let x = 1` 이라고 쓴 변수에 다음 줄에서 `x = "hello"` 를 넣어도 된다. 자유롭지만, 그 자유 때문에 "여기에는 숫자가 들어올 것"이라는 약속이 코드 어디에도 적히지 않는다. 약속이 깨지는 순간은 코드를 실행했을 때 비로소 드러난다.

TypeScript는 그 약속을 코드에 적게 한다. `let x: number = 1` 이라고 쓰면 `x = "hello"` 는 편집기에서 빨간 줄이 그어지고, 컴파일이 막힌다. 서버가 사용자 요청을 받기 전에, 내 자리에서 미리 잡힌다.

TS는 JS의 상위집합이다 — 모든 JS 코드는 그대로 유효한 TS 코드다. TS는 JS에 타입 표기(`: number`)와 인터페이스·제네릭 같은 도구를 더하며, 실행 직전에 그 타입 표기를 전부 떼어낸 순수 JS로 변환된다. 그래서 "JS를 알면 TS의 절반은 이미 안다"는 말이 성립한다.

tsx 가 변환과 실행을 한 번에 처리한다 — 원래는 `.ts` 를 `tsc` 로 `.js` 로 컴파일한 뒤 `node` 로 돌려야 한다. `tsx` 는 그 두 단계를 묶어 `.ts` 파일을 곧장 실행한다. 학습 중에는 `tsx <파일명>` 한 줄이면 충분하다.

준비

이 장의 `package.json` 은 의존성이 단 두 개다.

```
{
  "dependencies": {
    "dotenv": "^16.4.5",
    "reflect-metadata": "^0.2.2"
  }
}
```

- `reflect-metadata` — 데코레이터가 메타데이터를 다룰 때 쓴다. 데코레이터 예제 맨 위의 `import "reflect-metadata"` 가 이 패키지다.
- `dotenv` — `.env` 파일의 환경변수를 불러온다. Node 통합 예제에서 포트·호스트를 외부 설정으로 빼는 데 쓴다.

`tsconfig.json` 도 짧다.

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "CommonJS",
    "experimentalDecorators": true
  }
}
```

`experimentalDecorators: true` 가 핵심이다. 데코레이터는 아직 표준 진행 단계라, 이 옵션을 켜야

`@classDecorator` 같은 문법이 동작한다. 데코레이터 예제를 돌리려면 반드시 필요하다.

실행은 이렇게 한다.

```
npm install          # dotenv, reflect-metadata 설치
npm install -g tsx    # TS 실행기 (또는 매번 npx tsx 사용)

node <파일명>         # JS 예제
tsx <파일명>          # TS 예제
```

이제 두 묶음을 차례로 본다.

1부 · JavaScript

값, 변수, 비교, 분기 — JavaScript의 토대

출력과 문자열, 연산자

가장 먼저 `console.log` 다. 코드가 무슨 값을 만들었는지 눈으로 확인하는 창이다. 이 장의 예제는 모두 결과를 `console.log` 로 찍어 보여 준다.

```
console.log("Hello world");
console.log(`hello ${"world"}`);
```

작은따옴표·큰따옴표 문자열과 백틱(```) 템플릿 문자열의 차이가 핵심이다. 백틱 안에서는 `${. }` 로 값을 끼워 넣을 수 있다. 문자열 이어 붙이기(`"a" + "b"`)보다 읽기 쉬워서, 앞으로 거의 모든 출력에 이 형태를 쓴다.

```
console.log("this is \n string "); // \n 줄바꿈
console.log("this is \t string");   // \t 탭
console.log("this is \\ string");   // \\ 역슬래시 한 개
console.log(`52 + 45 = ${52 + 45}`); // 템플릿 안에서 계산
```

`\n`, `\t`, `\\` 는 이스케이프다. 역슬래시 뒤 한 글자가 특수 문자를 뜻한다. 역슬래시 자체를 찍으려면 `\\` 처럼 두 번 쓴다. 그리고 `${52 + 45}` 안에는 값뿐 아니라 식도 들어가 97 로 계산된다.

연산자도 한 번에 훑는다.

```
console.log(2 + 3 * 5); // 17 - 곱셈 먼저
console.log(10 % 5); // 0 - 나머지
console.log(45 = 52); // false
console.log(52 >= 52 && 52 > 52); // false - 둘 다 참이어야
console.log(52 >= 52 || 52 > 52); // true - 하나만 참이면
```

`%` (나머지)와 `&&` / `||` (논리 AND/OR)는 뒤에서 짝수 판별, 조건 분기에 끊임없이 등장한다.

변수 선언과 타입 변환

```
let pi;
console.log(pi); // undefined - 선언만 하고 값을 안 넣으면
pi = 3.141592;
console.log(pi);
```

`let pi` 처럼 값 없이 선언하면 그 변수는 `undefined` 다. "비어 있음"을 뜻하는 특별한 값이다.

```
let num = 0;
console.log(`num++ : ${num++}`); // 후위: 현재 값 0을 쓴 뒤 증가
console.log(`++num : ${++num}`); // 전위: 먼저 증가한 뒤 그 값을 씀
```

`num++` (후위)와 `++num` (전위)의 차이는 "값을 쓰는 시점"이다. 후위는 현재 값을 먼저 쓰고 나서 1을 더하고, 전위는 먼저 1을 더한 뒤 그 값을 쓴다. 헷갈리면 증감을 한 줄에서 같이 쓰지 말고 따로 떼는 게 안전하다.

타입 변환은 두 종류다. 명시적 변환과, JS가 알아서 하는 자동 변환.

```
console.log(Number("45")); // 45 - 문자열을 숫자로
console.log(Number("hello")); // NaN - 변환 실패
console.log("25" + 125); // "25125" - 문자열 + 숫자 → 이어 붙임
console.log("25" - 125); // -100 - 문자열 - 숫자 → 숫자로 변환 후 뺌
```

여기가 JS 초심자가 가장 많이 데이는 곳이다. `+` 는 한쪽이 문자열이면 이어 붙이기로 동작하고, `-` 는 그런 규칙이 없어 양쪽을 숫자로 바꾼다. 같은 "25" 인데 연산자에 따라 결과가 "25125" 와 -100 으로 갈린다. 이런 모호함을 TS가 잡는다.

```
console.log(parseInt("123abc")); // 123 - 앞의 숫자만 읽고 멈춤
console.log(parseInt("abc123")); // NaN - 시작이 숫자가 아니면 실패
console.log(parseInt("FF", 16)); // 255 - 16진수로 해석
```

`parseInt` 는 문자열 **앞쪽부터** 숫자를 읽다가 숫자가 아닌 글자를 만나면 멈춘다. `Number` 는 전체가 숫자여야 하지만 `parseInt` 는 `"123abc"` 도 `123` 으로 받아들인다. 두 번째 인자로 진법도 지정한다.

타입 확인과 동등 비교

```
console.log(typeof 10);           // number
console.log(typeof "hello");      // string
console.log(typeof function () {}); // function
let beta;
console.log(typeof beta);         // undefined
```

`typeof` 는 값의 타입을 문자열로 돌려준다. JS에는 타입 표기가 없으니, 실행 중에 타입을 확인하려면 이게 거의 유일한 수단이다. (TS 타입 가드의 출발점이기도 하다.)

이 장에서 가장 중요한 한 줄이 여기 있다.

```
console.log(`50 = "50" : ${50 = "50"}`); // true - 타입 변환 후 비교
console.log(`50 === "50" : ${50 === "50"}`); // false - 타입까지 비교
console.log(`0 = "" : ${0 = ""}`); // true
console.log(`0 === "" : ${0 === ""}`); // false
```

`=` 는 비교 전에 **타입을 맞춘다**. 그래서 숫자 `50` 과 문자열 `"50"` 이 같다고 나오고, `0` 과 빈 문자열 `""` 도 같다고 나온다. 이게 버그의 온상이다. `===` 는 타입이 다르면 무조건 `false` 다. **현업에서는 `===` 만 쓴다.** `=` 는 "왜 이게 같지?" 하는 함정을 만든다.

```
const test = "변경 불가";
// test = "hello"; // 에러: const는 재할당 불가
{
  const x = "block";
  console.log(x); // block
}
console.log(x); // 에러: x는 블록 밖에서 안 보임
```

`const` 는 한 번 정한 값을 다시 못 바꾼다. 그리고 `{ }` 블록 안에서 선언한 변수는 그 블록 안에서만 보인다(블록 스코프). 이 스코프 개념은 뒤에서 클로저로 다시 깊게 다룬다.

조건문과 단축 평가

`if` / `else if` / `else`, 삼항 연산자, `switch` 는 다른 언어와 거의 같다.

```
const timeOfDay = hour < 12 ? "오전" : "오후";
```

삼항 연산자 조건 `? A : B` 는 "조건이 참이면 A, 아니면 B"를 **값으로** 돌려준다. 간단한 분기를 한 줄로 줄일 때 쓴다.

더 눈여겨볼 건 단축 평가와 기본값 패턴이다.

```
const name = "";
const displayName = name || "익명"; // "" 는 falsy → "익명"
console.log(`환영합니다, ${displayName}님!`);

const userInput = null;
const result = userInput ?? "기본값"; // null/undefined일 때만 "기본값"
```

`||` 는 왼쪽이 거짓 같은 값(**falsy**)이면 오른쪽을 쓴다. `""`, `0`, `null`, `undefined`, `NaN` 이 falsy다. 그래서 `name` 이 빈 문자열이면 `"익명"` 이 된다.

`??` (nullish 병합)는 한 칸 더 엄격하다. `null` 이나 `undefined` 일 때만 오른쪽을 쓴다. `0` 이나 `""` 은 멀쩡한 값으로 본다. "숫자 0도 유효한 값"인 상황에서는 `||` 대신 `??` 를 써야 0이 기본값으로 덮이지 않는다. Express 핸들러에서 `author || "익명"` 같은 기본값 처리가 바로 이 패턴이다.

배열, 반복, 함수

배열과 반복문

```
let arr = [5, 23, "hello", true, "world", -9];
console.log(arr[2]); // hello - 0부터 센다
```

JS 배열은 한 배열에 숫자·문자열·불리언을 섞어 담을 수 있다. 자유롭지만, "이 배열엔 숫자만 들어 있겠지"라는 가정이 깨지기 쉽다. TS의 `number[]` 가 이걸 막는다.

반복문은 네 가지 형태가 나온다.

```
for (let i = 0; i < arr.length; i++) {.. } // 고전적 for
for (i in arr) {.. } // 인덱스(키) 순회 → "0", "1", "2".
for (e of arr) {.. } // 값 순회 → 5, 23, "hello".
```

`for.in` 은 인덱스를, `for.of` 는 값을 준다. 배열을 순회할 때는 보통 값이 필요하므로 `for.of` 가 자연스럽다. (`for.in` 은 원래 객체의 키를 도는 용도다 — 객체를 다룰 때 다시 쓴다.)

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const evenNumbers = numbers.filter((num) => num % 2 === 0);
console.log("짝수:", evenNumbers); // [2, 4, 6, 8, 10]
```

마지막 줄에 `filter` 가 미리 등장한다. `num % 2 === 0` (2로 나눈 나머지가 0)인 요소만 남긴다. 반복문을 직접 돌리는 대신 이런 **배열 메서드**로 거르는 방식은 배열 고차 함수에서 본격적으로 다룬다.

함수 세 가지와 콜백

같은 덧셈 함수를 세 가지 방법으로 쓴다.

```
let add1 = function (x, y) { return x + y; }; // 익명 함수
function add2(x, y) { return x + y; } // 선언 함수
let add3 = (x, y) => x + y; // 화살표 함수
```

셋 다 결과는 같다. 화살표 함수($\Rightarrow$)가 가장 짧고, 식이 한 줄이면 `return` 과 중괄호를 생략할 수 있다. 요즘 코드는 화살표 함수가 기본이고, 특히 다음에 나오는 콜백에서 빛난다.

```
function tentimes(callback) {
  for (let i = 0; i < 10; i++) {
    callback(); // 받은 함수를 열 번 실행
  }
}
tentimes(function () {
  console.log("call function");
});
```

함수를 다른 함수에 인자로 넘기는 것이 콜백이다. `tentimes` 는 "무엇을 할지"는 모른 채 "열 번 반복한다"는 틀만 갖고, 실제 할 일은 넘겨받은 `callback` 이 정한다. `filter((num) => ...)` 도 같은 구조다 — `filter` 는 "거른다"는 틀만 갖고, 조건은 콜백이 정한다.

```
setTimeout(function () {
  console.log('1초 후 호출');
}, 1000);
```

`setTimeout` 은 콜백을 1초 뒤에 실행한다. 지금 당장이 아니라 나중에 실행되는 이 "비동기" 흐름이 곧 에러 처리와 비동기 주제로 이어진다.

객체와 클래스

```
let personInfo = {
  name: "lee",
  age: 15,
  hobby: ["fishing", "rc", "star"],
};
console.log(personInfo["name"]); // 대괄호 접근
console.log(personInfo.age);     // 점 접근
for (let key in personInfo) {
  console.log(`${key} : ${personInfo[key]}`); // 객체 키 순회
}
```

객체는 **키: 값** 쌍의 묶음이다. 값을 꺼낼 때 `obj.age` (점)와 `obj["name"]` (대괄호) 둘 다 된다. 키가 변수에 담겨 있을 때는 대괄호를 쓴다. 객체의 키를 모두 돌 때는 `for.in` 이 제격이다.

```
let personInfoWithMethod = {
  age: 15,
  addAge: function () {
    this.age = this.age + 1; // this = 이 객체 자신
  },
};
personInfoWithMethod.addAge();
```

객체 안의 함수를 **메서드**라 한다. 메서드 안의 `this` 는 그 메서드를 호출한 객체를 가리킨다. 여기서는 `personInfoWithMethod` 자신이다.

객체가 많아지면 틀을 만드는 게 낫다. 그게 **클래스**다.

```
class PersonInfo {
  constructor(name, age, address, hobby) {
    this.name = name;
    this.age = age;
  }
  addAge(age) { this.age = this.age + age; }
  getAge() { return this.age; }
}
let p = new PersonInfo("lee", 12, "seoul", ["rc", "star"]);
```

`constructor` 는 `new` 로 객체를 만들 때 한 번 실행되어 초기값을 채운다. `class` 는 "같은 모양의 객체를 찍어내는 틀"이다.

```
class Employee extends PersonInfo {
  constructor(name, age, address, hobby, salary) {
    super(name, age, address, hobby); // 부모 생성자 호출
    this.salary = salary;
  }
}
class MathUtils {
  static add(a, b) { return a + b; } // 인스턴스 없이 호출
}
console.log(MathUtils.add(10, 20)); // 30
```

`extends` 는 상속이다. `Employee` 는 `PersonInfo` 의 모든 것을 물려받고 `salary` 만 더한다. `super(.)` 는 부모의 생성자를 부른다. `static` 메서드는 객체를 안 만들고 `클래스명.메서드()` 로 바로 부른다. 이 클래스·상속·`static` 개념은 NestJS의 서비스·컨트롤러에서 그대로 쓰인다.

에러 처리와 비동기 3단계

이 한 주제에 비동기의 역사가 다 들어 있다. 콜백 → Promise → `async/await` 순서로 같은 일을 점점 깔끔하게 푼다.

```
try {
  const err = new Error("This is Error");
  err.name = "My Error";
  throw err;
} catch (e) {
  console.log(`name:${e.name}, message: ${e.message}`);
} finally {
  console.log(`finally called`);
}
```

`try` 에서 에러가 나면(`throw`) 즉시 `catch` 로 점프한다. `finally` 는 에러가 났든 안 났든 **항상** 실행된다. 자원 정리(파일 닫기 등)에 쓴다.

이제 "1초 뒤에 데이터가 온다"를 세 방식으로 푼다.

```
// 1단계: 콜백
function fetchData(callback) {
  setTimeout(() => {
    callback("서버에서 받은 데이터"); // 끝나면 콜백 호출
  }, 1000);
}
fetchData((data) => console.log("콜백으로 받은 데이터:", data));
```

콜백은 동작하지만, 비동기 작업이 줄줄이 이어지면 콜백 안에 콜백이 겹쳐 들어가 읽기 힘들어진다(이른바 콜백 지옥).

```
// 2단계: Promise
function fetchDataPromise() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = true;
      if (success) resolve("서버에서 받은 데이터");
      else reject("데이터 요청 실패");
    }, 1000);
  });
}
fetchDataPromise()
  .then((data) => console.log("프라이미스로 받은 데이터:", data))
  .catch((error) => console.error("에러:", error));
```

Promise는 "지금은 없지만 나중에 결과가 올 약속"이다. 성공하면 `resolve(값)`, 실패하면 `reject(이유)` 를 부른다. 받는 쪽은 `.then` (성공)과 `.catch` (실패)로 잇는다. 콜백을 중첩하는 대신 **연결**로 퍼낸다.

```
// 3단계: async/await
async function getData() {
  try {
    const data = await fetchDataPromise(); // 결과가 올 때까지 기다림
    console.log("async/await로 받은 데이터:", data);
  } catch (error) {
    console.error("에러:", error);
  }
}
```

`async/await` 는 Promise를 동기 코드처럼 보이게 한다. `await` 는 "이 Promise가 끝날 때까지 기다렸다가 결과를 받아라"는 뜻이고, 에러는 평범한 `try/catch` 로 잡는다. `.then` 체인보다 위에서 아래로 읽혀서, 현업 비동기 코드는 거의 다 이 형태다. Express·NestJS의 DB 호출이 전부 `await` 다.

핵심 — 콜백·Promise·async/await는 같은 일을 다르게 적은 것이다. async/await가 가장 읽기 좋지만, 그 밑에는 Promise가 있고 Promise 밑에는 콜백이 있다. 셋의 관계를 알면 남의 코드를 읽을 때 막히지 않는다.

구조 분해와 스프레드/레스트

구조 분해 할당

배열이나 객체에서 값을 한 번에 꺼내 변수에 담는 문법이다.

```
const fruits = ["사과", "바나나", "오렌지"];
const [firstFruit, secondFruit, thirdFruit] = fruits; // 순서대로

const student = { name: "김철수", age: 20, grade: "A" };
const { name, age, grade } = student; // 키 이름으로
```

배열은 순서로, 객체는 키 이름으로 꺼낸다. `const { name } = student` 는 `const name = student.name` 을 짧게 쓴 것이다. Express에서 `const { title, content } = req.body` 로 본문을 꺼내는 게 바로 이 문법이다.

```
const { name: studentName, age: studentAge } = student; // 다른 이름으로
const { name: personName, age: personAge = 25 } = person; // 없으면 기본값 25
```

`name: studentName` 은 " `name` 을 꺼내되 `studentName` 이라는 변수에 담아라"다. `= 25` 는 그 키가 없을 때 쓸 기본값이다.

```
function printStudentInfo({ name, age, grade = "B" }) {
  console.log(`- 이름: ${name}, 나이: ${age}, 성적: ${grade}`);
}
printStudentInfo(student);
```

매개변수 자리에서 바로 구조 분해할 수도 있다. 객체 하나를 받아 그 안의 필드를 꺼내 쓰는 이 패턴은 함수 인자가 많을 때 특히 깔끔하다.

```
const { id, info: { name: userName, address: { city, street } } } = user;
```

중첩 객체도 같은 모양을 겹쳐 파고든다. 깊은 곳의 값까지 한 줄로 꺼낸다.

스프레드와 레스트

`..` 은 위치에 따라 정반대 일을 한다. 펼치면 스프레드, 모으면 레스트다.

```
const numbers = [1, 2, 3];
const more = [4, 5, 6];
const combined = [...numbers, ...more]; // [1,2,3,4,5,6] - 펼쳐서 합침

const original = [10, 20, 30];
const copy = [...original]; // 복사본 (원본과 별개)
copy.push(40); // original은 그대로
```

`[...numbers]` 는 배열을 펼쳐서 새 배열에 풀어 넣는다. 합치기와 복사가 한 번에 된다. 객체도 마찬가지다.

```
const user = { name: "홍길동", age: 25 };
const updated = { ..user, age: 26, email: "hong@example.com" };
// age는 덮어쓰고, email은 새로 추가
```

`{..user, age: 26}` 은 `user` 를 펼친 뒤 `age` 만 새 값으로 덮는다. 뒤에 적은 게 이긴다. 이 "복사 후 일부만 변경"이 다음 예제로 이어진다.

```
function sum(.nums) {           // 인자를 전부 배열로 모음
  return nums.reduce((acc, n) => acc + n, 0);
}
console.log(sum(1, 2, 3, 4, 5)); // 15

const [first, second, ...others] = [10, 20, 30, 40, 50];
console.log(others); // [30, 40, 50] - 나머지를 모음
const { name, ...etc } = { name: "김철수", age: 30, city: "서울" };
console.log(etc); // { age: 30, city: '서울' }
```

함수 매개변수나 구조 분해 자리에서... 은 반대로 남은 것을 모은다(레스트). 같은... 인데 "펼치는 자리"냐 "모으는 자리"냐로 구분된다.

```
const state = { count: 0, user: { name: "홍길동", age: 25 } };
const nextState = {
  ...state,
  count: state.count + 1,
  user: { ..state.user, age: 26 },
};
```

마지막은 불변 업데이트다. 원본 `state` 를 건드리지 않고, 펼쳐서 복사한 새 객체에 바뀐 값만 덮는다. React-Redux에서 상태를 바꾸는 표준 방식이고, 원본을 안 망가뜨려 추적이 쉽다.

스코프, 호이스팅, 클로저

```
function testVarScope() {
  if (true) {
    var funcVar = "if 안에서 선언";
  }
  console.log(funcVar); // 보임! var는 함수 전체 스코프
}
```

`var` 는 블록(`if`, `for`)을 무시하고 함수 전체에서 보인다. 의도치 않은 곳에서 변수가 살아 있어 버그를 만든다. 그래서 요즘은 블록 스코프인 `let` / `const` 만 쓴다.

```
console.log("선언 전 hoisted:", hoisted); // undefined
var hoisted = "값 할당";

try {
  console.log(notHoisted); // ReferenceError
  let notHoisted = "값";
} catch (e) { console.log("에러:", e.message); }
```

`var` 는 선언이 위로 끌어올려져(호이스팅) 선언 전에 접근해도 `undefined` 로 나온다 — 버그를 숨긴다. `let` / `const` 는 선언 전 접근을 **에러**로 막는다(TDZ, Temporal Dead Zone). 에러로 막는 쪽이 안전하다.

이 주제의 핵심은 클로저다.

```
function createCounter() {
  let count = 0; // 외부 함수의 변수
  return function () {
    count += 1; // 내부 함수가 외부 변수를 기억
    return count;
  };
}
const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
const counter2 = createCounter();
console.log(counter2()); // 1 (counter와 별개)
```

클로저는 함수가 자기 바깥의 변수를 기억하는 현상이다. `createCounter` 가 끝나도 `count` 는 사라지지 않고, 돌려준 내부 함수가 계속 붙들고 있다. `counter` 와 `counter2` 는 각자 ****독립된 `count` ****를 가진다. 이게 클로저의 핵심이다.

```
function createBankAccount(initialBalance) {
  let balance = initialBalance; // 외부에서 직접 못 건드림
  return {
    deposit(amount) { balance += amount; return balance; },
    withdraw(amount) {
      if (amount > balance) throw new Error("잔액 부족");
      balance -= amount; return balance;
    },
    getBalance() { return balance; },
  };
}
const account = createBankAccount(1000);
console.log(account.deposit(500)); // 1500
console.log(account.balance); // undefined - 비공개!
```

클로저로 **비공개 상태**를 만든다. `balance` 는 함수 안에 갇혀 있어 `account.balance` 로는 못 본다. 오직 `deposit` / `withdraw` / `getBalance` 를 통해서만 접근된다 — 캡슐화다. 외부에서 잔액을 멋대로 바꾸지 못하게 막는다.

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var i:", i), 10); // 3, 3, 3
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log("let j:", j), 20); // 0, 1, 2
}
```

유명한 함정이다. `var i` 는 반복 전체가 하나의 `i` 를 공유해서, 비동기 콜백이 실행될 때쯤엔 모두 `3` 이 된다. `let j` 는 매 반복마다 **새 `j` **가 생겨 각자 `0, 1, 2` 를 기억한다. `var` 를 피하고 `let` 을 쓰는 또 하나의 이유다.

배열 고차 함수

여기서는 실무에서 가장 많이 쓰는 배열 메서드를 모아 본다. 공통점은 **콜백을 받아** 각 요소를 처리한다는 것이다(앞서 본 콜백을 기억하자).

```
const users = [
  { id: 1, name: "홍길동", age: 25, score: 85 },
  { id: 2, name: "김철수", age: 30, score: 92 },
  //..
];

const youngUsers = users.filter((user) => user.age < 30); // 거른다
const userNames = users.map((user) => user.name); // 변환한다
const totalScore = users.reduce((sum, user) => sum + user.score, 0); // 누적
```

- **filter** — 조건이 참인 요소만 남긴 **새 배열**. (30세 미만만)
- **map** — 각 요소를 변환한 **새 배열**. (객체 배열 → 이름 배열)
- **reduce** — 배열을 **하나의 값**으로 접는다. `sum` 에 점수를 계속 더해 합계를 낸다. 두 번째 인자 `0` 이 시작값이다.

```
const highScoreUser = users.find((user) => user.score > 90); // 첫 요소
const hasPerfectScore = users.some((user) => user.score === 100); // 하나라도?
const allAdults = users.every((user) => user.age ≥ 20); // 전부?
const sortedByAge = [...users].sort((a, b) => a.age - b.age); // 정렬
```

- **find** — 조건에 맞는 **첫 요소 하나**. (없으면 `undefined`)
- **some** — 하나라도 만족하면 `true`.
- **every** — 전부 만족해야 `true`.
- **sort** — `a.age - b.age` 가 음수면 `a` 를 앞에 둔다(오름차순). `[...users]` 로 복사본을 정렬하는 게 중요하다. `sort` 는 원본을 바꾸기 때문이다.

```
const userMap = users.reduce((acc, user) => {
  acc[user.id] = user;
  return acc;
}, {}); // { 1: {..}, 2: {..}, ... }
```

`reduce` 는 합계뿐 아니라 객체로 모으는 데도 쓴다. 빈 객체 `{}` 에서 시작해 `id` 를 키로 사용자를 꽂아 넣는다. `id` 로 빠르게 찾는 조희용 맵이 된다.

메서드	받는 것	돌려주는 것
<code>filter</code>	조건 콜백	통과한 요소들의 새 배열
<code>map</code>	변환 콜백	변환된 요소들의 새 배열
<code>reduce</code>	누적 콜백 + 시작값	하나로 접은 값
<code>find</code>	조건 콜백	맞는 첫 요소(없으면 <code>undefined</code>)
<code>some</code> / <code>every</code>	조건 콜백	<code>true</code> / <code>false</code>
<code>sort</code>	비교 콜백	정렬된 배열(원본 변경)

이 메서드들은 `for` 반복문보다 무엇을 하는지가 한눈에 드러난다. Express에서 게시물을 거르고(`filter`), DB 결과를 응답 형태로 바꾸는(`map`) 코드가 전부 이것이다.

2부 · TypeScript

여기서부터는 같은 개념에 타입이 붙는다. JS에서 본 함수·객체·클래스가 어떻게 안전해지는지에 집중한다.

기본 타입 — TypeScript의 타입 표기

```
let isDone: boolean = false;
let decimal: number = 6;
let color: string = "blue";
let list: number[] = [1, 2, 3];
let tuple: [string, number] = ["hello", 10];
```

변수 이름 뒤 `:` 타입 이 타입 표기다. `let color: string` 이라고 하면 `color` 에는 문자열만 들어간다. `number[]` 는 "숫자만 담은 배열", `[string, number]` 는 튜플 — 자리마다 타입이 정해진 고정 길이 배열이다. JS에서 배열에 아무거나 섞던 것과 대비된다.

```
enum Color { Red, Green, Blue }
let favoriteColor: Color = Color.Blue; // 2 (0,1,2 순서)
```

`enum` 은 이름 붙인 상수 묶음이다. `Color.Blue` 가 숫자 `2` 에 대응한다. "정해진 값 중 하나"를 표현할 때 쓴다.

```
let notSure: any = 4;
notSure = "string"; // any는 뭐든 허용 - 타입 검사 포기
function warnUser(): void { console.log("."); } // 반환값 없음
function error(message: string): never { throw new Error(message); } // 끝나지 않음
```

- `any` — 아무 타입이나 허용한다. 편하지만 타입 검사를 꺼버리는 것이라 꼭 필요할 때만 쓴다. TS를 쓰는 의미를 없앤다.
- `void` — 반환값이 없는 함수의 반환 타입.
- `never` — 정상적으로 끝나지 않는 함수(에러를 던지거나 무한 루프).

```
let someValue: any = "this is a string";
let strLength: number = (someValue as string).length; // 타입 단언

let unionType: string | number = "hello"; // 유니온: 둘 중 하나
unionType = 42; // 둘 다 허용
```

`as string` 은 타입 단언 — "이 값을 문자열로 취급하라"고 컴파일러에 알린다. `string | number` 는 유니온 타입 — "문자열이거나 숫자"다. 이 유니온은 판별 유니온과 타입 가드에서 핵심이 된다.

인터페이스와 타입 별칭

객체의 모양에 이름을 붙이는 두 가지 방법이다.

```
interface User {
  name: string;
  age: number;
}
const user: User = { name: "홍길동", age: 25 };
```

`interface User` 는 " name (문자열)과 age (숫자)를 가진 객체"라는 약속이다. 이제 `User` 타입 변수에는 그 모양을 지킨 객체만 들어간다. 필드를 빠뜨리거나 타입이 어긋나면 컴파일 에러다.

```
interface ColorConfig {
  color?: string; // ? - 있어도 되고 없어도 됨
  width?: number;
}
interface ReadonlyPoint {
  readonly x: number; // 한 번 정하면 못 바꿈
  readonly y: number;
}
```

? 는 선택적 프로퍼티다. `color?` 는 없어도 된다. `readonly` 는 생성 후 변경을 막는다. JS의 `const` 가 변수에 거는 제약을, `readonly` 는 객체의 한 필드에 건다.

```
interface SearchFunc {
  (source: string, subString: string): boolean; // 함수 타입
}
interface ClockInterface {
  currentTime: Date;
  setTime(d: Date): void;
}
class Clock implements ClockInterface { // 인터페이스를 구현
  currentTime: Date = new Date();
  setTime(d: Date) { this.currentTime = d; }
}
```

인터페이스로 함수의 모양도 정의한다. `implements` 는 "이 클래스는 이 인터페이스의 약속을 전부 지킨다"는 선언이다. 약속을 어기면 컴파일러가 잡는다. NestJS에서 이 패턴을 자주 본다.

```
type PointType = { x: number; y: number }; // 타입 별칭

interface Shape { color: string; }
interface Square extends Shape { sideLength: number; } // 확장
```

`type` 은 타입 별칭 — 타입에 이름을 붙인다. 인터페이스와 비슷하지만 유니온(`string | number`)에도 이름을 붙일 수 있어 더 유연하다. `extends` 는 인터페이스 상속 — `Square` 는 `Shape` 의 `color` 에 `sideLength` 를 더한다.

interface와 type, 언제 뭘? — 객체나 클래스의 모양에는 `interface` , 유니온·튜플 등 그 외에는 `type` 을 쓰는 게 일반적인 관례다. 둘 다 되는 경우가 많으니 처음엔 크게 고민하지 않아도 된다.

함수와 제네릭

JS 함수에 타입이 붙으면 인자와 반환값이 명확해진다.

```
function add(x: number, y: number): number { return x + y; }
function buildName(firstName: string, lastName?: string): string {.. } // 선택적
function greet(name: string, greeting: string = "안녕하세요"): string {.. } // 기본값
function sum(. numbers: number[]): number {.. } // 나머지 매개변수
```

`x: number` (인자 타입), `: number` (반환 타입)가 한눈에 들어온다. `lastName?` 는 선택적 인자, `greeting = !. "` 은 기본값, `.. numbers: number[]` 는 레스트다 — JS에서 본 것에 타입만 붙었다.

```
function process1(x: number): number;
function process1(x: string): string;
function process1(x: any): any {
  if (typeof x === "number") return x * 2;
  return `processed: ${x}`;
}
```

오버로드다. 위 두 줄은 "숫자를 넣으면 숫자가, 문자열을 넣으면 문자열이 나온다"는 선언이고, 실제 구현은 맨 아래 하나다. 입력 타입에 따라 출력 타입이 달라짐을 컴파일러에 알린다.

이 주제의 핵심은 제네릭이다.

```
function identity<T>(arg: T): T { return arg; }
console.log(identity<number>(42)); // T = number
console.log(identity<string>("hello")); // T = string
```

`<T>` 가 제네릭 — 타입을 나중에 정하는 자리표시자다. `identity` 는 "받은 그대로 돌려주되, 들어온 타입을 그대로 유지한다". `any` 로 받으면 타입이 뭉개지지만, `<T>` 는 호출할 때의 타입을 끝까지 기억한다. 타입 안전성을 잃지 않고 재사용한다.

```
function logValue<T extends string | number | boolean>(value: T): void {.. }
function merge<T extends object, U extends object>(obj1: T, obj2: U): T & U {
  return {.. obj1, .. obj2 };
}
function pair<T, U>(first: T, second: U): [T, U] { return [first, second]; }
```

`<T extends...>` 는 제약이다. `T` 를 아무거나가 아니라 "문자열·숫자·불리언 중 하나"로 좁힌다. `merge` 의 반환 `T & U` 는 인터섹션 — 두 타입을 합친 타입이다. `pair` 는 두 타입을 받아 튜플로 묶는다. 제네릭은 유틸리티 타입과 실전 코드에서 다시 만난다.

클래스, 접근 제어자, 추상 클래스

JS 클래스에 접근 제어자가 붙는 게 가장 큰 차이다.

```

class Animal {
  protected name: string;    // 자신 + 자식만 접근
  protected age: number;
  constructor(name: string, age: number) {
    this.name = name; this.age = age;
  }
  public move(distance: number = 0): void { // 어디서든 접근
    console.log(`${this.name} moved ${distance}m.`);
  }
}
class Dog extends Animal {
  private breed: string;    // 이 클래스 안에서만
  //...
  public getInfo(): string {
    return `${super.getInfo()}, 품종: ${this.breed}`;
  }
}

```

- `public` — 어디서든 접근(기본값).
- `private` — 그 클래스 안에서만. `Dog` 의 `breed` 는 밖에서 못 본다.
- `protected` — 그 클래스와 자식 클래스까지. `Animal` 의 `name` 을 `Dog` 가 쓸 수 있다.

앞서 클로저로 힘들게 만든 "비공개 상태"를, TS는 `private` 키워드 하나로 선언한다. 훨씬 명시적이다.

```

abstract class ShapeType {
  protected color: string;
  abstract getArea(): number;    // 구현 없음 - 자식이 채워야 함
  abstract getPerimeter(): number;
}
class Circle extends ShapeType {
  getArea(): number { return Math.PI * this.radius * this.radius; }
  getPerimeter(): number { return 2 * Math.PI * this.radius; }
}

```

추상 클래스는 그 자체로는 객체를 못 만든다(`new ShapeType()` 불가). `abstract getArea()` 는 "본문은 없고, 상속받는 클래스가 반드시 구현하라"는 빈 약속이다. 자식이 약속을 안 지키면 컴파일 에러다.

```
class Bird extends Animal implements Flyable { // 상속 + 구현 동시에
  fly(): void {.. }
}
class Person {
  private _name: string = "";
  get name(): string { return this._name; }
  set name(value: string) {
    if (value.length < 3) throw new Error("이름은 3자 이상이어야 합니다.");
    this._name = value;
  }
}
const person = new Person();
person.name = "홍길동"; // set 호출 → 검증 통과
```

`extends` (상속)와 `implements` (인터페이스 구현)를 동시에 쓸 수 있다. **getter/setter**는 필드처럼 보이지만 함수다. `person.name = "홍길동"` 이 평범한 대입 같아도 실제로는 `set` 이 호출되어 길이 검증을 거친다. 비공개 필드 `_name` 을 검증과 함께 노출하는 표준 패턴이다.

모듈과 네임스페이스

```
export interface User { name: string; age: number; }
export class UserService {
  private users: User[] = [];
  addUser(user: User): void { this.users.push(user); }
  getUsers(): User[] { return this.users; }
}
import { UserService as UserService2, User as User2 } from "./user-service";
```

`export` 는 다른 파일이 쓸 수 있게 **내보낸다**, `import` 는 가져온다. `as` 로 다른 이름을 붙일 수 있다. 코드를 파일 단위로 나눠 재사용하는 이 모듈 시스템이 모든 Node 프로젝트의 뼈대다. Express의 라우터 분리, NestJS의 모듈이 전부 이 `import / export` 다.

```
namespace Calculator {
  export interface Operation { calculate(a: number, b: number): number; }
  export class Add implements Operation {
    calculate(a: number, b: number): number { return a + b; }
  }
}
const add = new Calculator.Add();
console.log(add.calculate(10, 5)); // 15
```

네임스페이스는 관련된 것들을 한 이름 아래 묶는다. `Calculator.Add` 처럼 점으로 접근한다. 다만 요즘은 파일 모듈 (`import / export`)이 표준이라, 네임스페이스는 알아만 두고 새 코드에서는 잘 쓰지 않는다.

유니온과 판별 유니온

```
interface ASquare { kind: "square"; size: number; }
interface Rectangle { kind: "rectangle"; width: number; height: number; }
type GeometricShape = ASquare | Rectangle;

function getArea(shape: GeometricShape): number {
  switch (shape.kind) {
    case "square": return shape.size * shape.size;
    case "rectangle": return shape.width * shape.height;
  }
}
```

이게 ****판별 유니온(discriminated union)****이다. 핵심은 `kind: "square"` 처럼 각 타입에 고정된 문자열 표식을 두는 것이다. `GeometricShape` 는 둘 중 하나인데, `switch (shape.kind)` 로 분기하면 TS가 똑똑하게 추론한다. `case "square"` 안에서는 `shape` 가 `ASquare` 임을 알아 `shape.size` 를 허용하고, `shape.width` 를 쓰면 에러다.

JS에서 객체에 아무 속성이나 접근해 `undefined` 를 받던 것과 다르다. TS는 "지금 이 타입에 그 속성이 있는가"를 분기마다 따진다. 서로 다른 형태의 데이터를 한 함수에서 안전하게 다루는 핵심 기법이다.

타입 가드

런타임에 "이 값이 정말 이 타입인가"를 확인하고, 그 결과를 컴파일러에 알리는 방법이다.

```
interface Car { type: "car"; brand: string; model: string; }
interface Bike { type: "bike"; brand: string; isElectric: boolean; }
type Vehicle = Car | Bike;

function isCar(vehicle: Vehicle): vehicle is Car { // 타입 서술자
  return vehicle.type === "car";
}

function printVehicleInfo(vehicle: Vehicle): void {
  if (isCar(vehicle)) {
    console.log(`자동차: ${vehicle.brand} ${vehicle.model}`); // Car로 좁혀짐
  } else {
    console.log(`자전거: ${vehicle.brand} ${vehicle.isElectric ? "전기" : "일반"}`);
  }
}
```

반환 타입 자리의 `vehicle is Car` 가 ****타입 서술자(type predicate)****다. "이 함수가 `true` 를 돌려주면 그 인자는 `Car` 다"라고 컴파일러에 알린다. 그래서 `if (isCar(vehicle))` 블록 안에서는 `vehicle` 이 `Car` 로 좁혀져 `vehicle.model` 을 안전하게 쓴다.

```
function isUser(value: any): value is User {
  return (
    typeof value === "object" && value !== null &&
    typeof value.name === "string" && typeof value.age === "number"
  );
}
const userData = { name: "홍길동", age: 25 };
if (isUser(userData)) {
  console.log("User:", userData.name, userData.age);
}
```

외부에서 들어온 정체불명의 데이터(any)를 검사해 "진짜 User 모양인가"를 확인한다. `typeof` 로 각 필드의 타입을 일일이 따진다(앞서 본 `typeof` 가 여기서 쓰인다). API 응답이나 사용자 입력처럼 타입을 믿을 수 없는 데이터를 다룰 때 필수다.

```
let x = 3; // number로 추론
let arr = [1, 2, 3]; // number[]로 추론
const baseAnimal2: BaseAnimal = baseDog; // 하위 타입은 상위 타입에 할당 가능
```

타입을 안 적어도 TS가 값을 보고 추론한다. `let x = 3` 이면 `x` 는 `number` 다. 그리고 `BaseDog` 가 `BaseAnimal` 의 모든 필드를 가졌으면 `BaseAnimal` 자리에 넣을 수 있다(구조적 타이핑).

유틸리티 타입

기존 타입을 변형해 새 타입을 만드는 내장 도구다. 같은 모양을 매번 다시 쓰지 않게 한다.

```
interface User { name: string; age: number; email?: string; }

type PartialUser = Partial<User>; // 모든 필드가 선택적
type RequiredUser = Required<User>; // 모든 필드가 필수 (email?도 필수로)
type ReadonlyUser = Readonly<User>; // 모든 필드가 readonly
```

- **Partial<User>** — 모든 필드를 선택적(?)으로. 일부만 채워 수정하는 PATCH 요청에 딱 맞는다.
- **Required<User>** — 모든 필드를 필수로. `email?` 도 반드시 있어야 한다.
- **Readonly<User>** — 모든 필드를 읽기 전용으로.

```
type CatName = "molly" | "boris" | "maggie";
type Cats = Record<CatName, CatInfo>; // 키가 CatName, 값이 CatInfo인 객체
const cats: Cats = {
  molly: { age: 3, breed: "페르시아" },
  boris: { age: 5, breed: "러시아 블루" },
  maggie: { age: 2, breed: "삼" },
};
```

**** Record<K, T> ****는 "키가 **K**, 값이 **T** 인 객체" 타입을 만든다. **CatName** 에 정의한 세 이름이 전부 있어야 한다 — 하나라도 빠지면 에러다.

```
type UserNameAndAge = Pick<User, "name" | "age">; // 고른 필드만
type UserWithoutAge = Omit<User, "age">;          // 뺀 나머지
```

- **Pick<User, "name" | "age">** — 지정한 필드만 골라낸다.
- **Omit<User, "age">** — 지정한 필드만 빼고 나머지를 가져온다.

이 둘은 기존 타입에서 일부만 필요할 때 새 인터페이스를 다시 쓰지 않게 한다. 뒤의 실전 Todo 앱에서 **Omit<Todo, "id" | "createdAt">** 로 "생성 시점엔 아직 id가 없는 Todo"를 표현하는 게 실전 예다.

유틸리티	하는 일
Partial<T>	모든 필드를 선택적으로
Required<T>	모든 필드를 필수로
Readonly<T>	모든 필드를 읽기 전용으로
Record<K, T>	키 K , 값 T 인 객체 타입
Pick<T, K>	K 필드만 골라냄
Omit<T, K>	K 필드만 제외

데코레이터

이 예제는 맨 위에 `import "reflect-metadata"` 가 있고, `tsconfig.json` 의 `"experimentalDecorators": true` 가 켜져 있어야 동작한다.

데코레이터는 클래스-메서드-프로퍼티에 **@** 를 붙여 기능을 덧입히는 문법이다. NestJS가 이 데코레이터로 거의 모든 것을 표현하니, 여기서 원리를 봐 두면 좋다.

```
function classDecorator<T extends { new(...args: any[]): {} }>(constructor: T) {
    return class extends constructor {
        newProperty = "새로운 프로퍼티";
        hello = "안녕하세요";
    };
}
@classDecorator
class ExampleClass {
    constructor(public name: string) {}
}
const exampleClass = new ExampleClass("홍길동");
console.log((exampleClass as any).newProperty); // "새로운 프로퍼티"
```

클래스 데코레이터는 클래스를 받아 새 클래스를 돌려준다. `@classDecorator` 를 붙이면 `ExampleClass` 에 원래 없던 `newProperty` , `hello` 가 생긴다. 클래스 정의를 건드리지 않고 기능을 끼워 넣는다.

```
function log(target: any, propertyKey: string, descriptor: PropertyDescriptor) {
    const originalMethod = descriptor.value;
    descriptor.value = function(...args: any[]) {
        console.log(`메서드 ${propertyKey} 호출됨`);
        const result = originalMethod.apply(this, args);
        console.log(`메서드 ${propertyKey} 완료됨`);
        return result;
    };
    return descriptor;
}
class ExampleMethod {
    @log
    greet(name: string): string { return `안녕하세요, ${name}!`; }
}
```

메서드 데코레이터는 메서드의 동작을 감싼다. `@log` 를 붙인 `greet` 를 호출하면, 원래 동작 앞뒤로 로그가 찍힌다. 원래 메서드(`originalMethod`)를 보관해 두고, 그것을 호출하는 새 함수로 바꿔치기하는 구조다. "기존 코드를 안 고치고 동작을 추가"하는 전형적 패턴이다.

```
function format() {
  return function (target: any, propertyKey: string) {
    let value = target[propertyKey];
    Object.defineProperty(target, propertyKey, {
      get() { return value; },
      set(newVal: string) { value = newVal.toUpperCase(); }, // 대문자로 변환
      enumerable: true, configurable: true,
    });
  };
}
class ExampleProperty {
  @format()
  name: string = "홍길동";
}
const ep = new ExampleProperty();
ep.name = "kim";
console.log(ep.name); // "KIM" - set에서 대문자로
```

프로퍼티 데코레이터는 필드에 getter/setter를 끼워 넣는다. `@format()` 을 붙이면 `name` 에 값을 넣을 때마다 자동으로 대문자가 된다. 앞서 본 getter/setter를 데코레이터로 재사용할 수 있게 만든다.

왜 데코레이터를 배우나 — NestJS의 `@Controller()`, `@Get()`, `@Injectable()` 이 전부 이 데코레이터다. 지금은 "클래스/메서드/프로퍼티에 기능을 덧붙이는 문법"이라는 감만 잡으면 충분하다. 13장 이후 NestJS에서 본격적으로 쓴다.

타입과 함께 보는 비동기 (Promise, async/await)

앞서 JavaScript에서 본 비동기를 TS 타입과 함께 다시 본다.

```
function delay(ms: number): Promise<void> {
  return new Promise((resolve) => setTimeout(resolve, ms));
}
function fetchUser(id: number): Promise<{ id: number; name: string }> {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) resolve({ id, name: "홍길동" });
      else reject(new Error("유효하지 않은 ID"));
    }, 1000);
  });
}
```

JS의 Promise에 반환 타입이 붙었다. `Promise<void>` 는 "값 없이 끝나는 비동기", `Promise<{ id: number; name: string }>` 는 "그 모양의 객체를 돌려주는 비동기"다. 결과의 타입이 명시되니, 받는 쪽에서 `user.name` 을 쓸 때 자동완성과 검사가 된다.

```

async function getUser(id: number): Promise<void> {
  try {
    const user = await fetchUser(id); // user는 { id, name } 타입
    console.log("사용자:", user);
  } catch (error) {
    console.error("에러:", error);
  }
}

```

async/await 도 JS와 같다. await fetchUser(id) 의 결과 user 는 TS가 { id, name } 으로 안다.

```

async function fetchMultipleUsers(ids: number[]): Promise<void> {
  const promises = ids.map((id) => fetchUser(id));
  const users = await Promise.all(promises); // 모두 끝날 때까지
  console.log("모든 사용자:", users);
}

async function raceExample(): Promise<void> {
  const result = await Promise.race([fastPromise, slowPromise]); // 가장 빠른 하나
}

```

- **Promise.all** — 여러 비동기를 동시에 시작하고 전부 끝날 때까지 기다린다. 하나씩 await 하는 것보다 빠르다. 사용자 여러 명을 한꺼번에 가져올 때 쓴다.
- **Promise.race** — 여러 개 중 가장 먼저 끝난 하나의 결과만 받는다. 타임아웃 처리 등에 쓴다.

```

function processUser(id: number): Promise<string> {
  return fetchUser(id)
    .then((user) => { return user.name; })
    .then((name) => { return name.toUpperCase(); })
    .catch((error) => { throw error; });
}

```

.then 체이닝도 보여 준다. 앞 .then 의 반환값이 다음 .then 으로 흘러간다. 같은 일을 async/await 로도, .then 으로도 쓸 수 있다는 걸 한 파일에서 비교한다.

Node 모듈과 타입

Node 내장 모듈을 TS 타입과 함께 다루는 통합 예제다. fs, path, http, dotenv 를 쓴다.

```
import * as fs from "fs";
import * as path from "path";
import * as http from "http";
import * as dotenv from "dotenv";

const env = dotenv.config().parsed || {};
```

`import * as fs` 로 Node 내장 모듈(`fs` : 파일, `http` : 서버)과 `dotenv` 를 가져온다. `dotenv.config()` 는 `.env` 파일을 읽어 환경변수로 올린다. 비밀번호·포트 같은 설정을 코드 밖으로 빼는 표준 방식이다. (앞서 본 `dotenv` 의존성이 여기 쓰인다.)

```
class FileHandler {
  static async readFile(filePath: string): Promise<string> {
    return new Promise((resolve, reject) => {
      fs.readFile(filePath, "utf-8", (err, data) => {
        if (err) reject(err);
        else resolve(data);
      });
    });
  }
}
```

콜백 기반인 `fs.readFile` 을 **Promise**로 감싼다. 앞서 본 "콜백 → Promise" 변환을 실제 Node API에 적용한다. 이렇게 감싸 두면 바깥에서는 `await FileHandler.readFile(. )` 로 깔끔하게 쓴다.

```
class Server {
  private server: http.Server;
  private config: Config;
  constructor(config: Config) {
    this.config = config;
    this.server = http.createServer(this.handleRequest.bind(this));
  }
  private handleRequest(req: http.IncomingMessage, res: http.ServerResponse): void {
    if (req.url === "/" ) {
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ message: "안녕하세요!" }));
    } else if (req.url === "/users") {
      const users: User[] = [ /*.. */ ];
      res.end(JSON.stringify(users));
    } else {
      res.writeHead(404, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ error: "페이지를 찾을 수 없습니다." }));
    }
  }
}
```

Node 내장 `http` 모듈로 만든 **날것의 서버**다. `req.url` 로 경로를 직접 분기하고, 상태코드·헤더·본문을 손으로 다 지정한다. 4장에서 배울 Express가 바로 이 반복을 줄인다. 여기서 한 번 직접 만들어 보면 Express가 무엇을 대신하는지 또렷이 보인다. 이 서버를 4장의 Express와 이어서 보면 된다.

```
const config: Config = {
  port: parseInt(env.PORT || "3000", 10),
  host: env.HOST || "localhost",
};
const server = new Server(config);
server.start();
```

환경변수 `PORT` 가 있으면 그 값을, 없으면 `"3000"` 을 쓴다(앞서 본 `||` 기본값 패턴). 설정을 타입(`Config`)으로 고정하고 외부 값으로 채우는 실전 구조다.

실전 Todo 앱

마지막은 지금까지의 타입 도구를 한데 엮은 작은 앱이다. 인터페이스·제네릭·유틸리티 타입·클래스·싱글톤이 함께 등장한다.

```
interface Todo {
  id: number; title: string; completed: boolean; createdAt: Date;
}
interface ApiResponse<T> { // 제네릭 인터페이스
  data: T; status: number; message: string;
}
```

`ApiResponse<T>` 가 **제네릭 인터페이스**다. `data` 의 타입을 나중에 정한다. `ApiResponse<Todo[]>` 면 `data` 가 `Todo` 배열, `ApiResponse<Todo>` 면 단일 `Todo`다. API 응답의 공통 껍데기(상태·메시지)는 고정하고 알맹이만 바꾸는 패턴이다. 실무 API 응답 타입이 거의 이 모양이다.

```
class Logger {
  static info(message: string): void {
    console.log(`[INFO] ${new Date().toISOString()} - ${message}`);
  }
  static error(message: string, error?: Error): void {
    console.error(`[ERROR] ${new Date().toISOString()} - ${message}`);
    if (error) console.error(error.stack);
  }
}
```

`static` 메서드만 가진 로거다. 객체를 안 만들고 `Logger.info(. )` 로 어디서나 호출한다(앞서 본 `static`). 로그 형식을 한곳에 모은다.

```

async createTodo(todo: Omit<Todo, "id" | "createdAt">): Promise<ApiResponse<Todo>> {
  const response = {
    data: { id: 1, ...todo, createdAt: new Date() },
    status: 201, message: "생성됨",
  };
  return response;
}

```

`Omit<Todo, "id" | "createdAt">` 가 정확하게 쓰였다. 새 **Todo**를 만들 때는 **id** 와 **createdAt** 이 아직 없다 — 그건 서버가 부여한다. 그래서 입력 타입에서 그 둘을 뺀다. 앞서 본 `Omit` 이 실전에서 왜 필요한지 보여 주는 대목이다.

```

class Config {
  private static instance: Config;
  private constructor() { this.settings = { /*.. */ }; }
  static getInstance(): Config {
    if (!Config.instance) Config.instance = new Config();
    return Config.instance;
  }
  get<T>(key: string): T { return this.settings[key] as T; }
}

```

싱글턴 패턴이다. `constructor` 를 `private` 으로 막아 외부에서 `new Config()` 를 못 하게 하고, `getInstance()` 로만 단 하나의 인스턴스를 얻게 한다. 설정처럼 앱 전체에서 하나만 있어야 하는 객체에 쓴다. `get<T>` 는 제네릭이라 꺼내는 값의 타입을 호출 시점에 지정한다. NestJS의 의존성 주입도 내부적으로 이런 "하나만 만들어 공유"하는 개념을 따른다.

```

class TodoApp {
  private api: TodoApi;
  private database: Database;
  constructor() {
    this.config = Config.getInstance();
    this.api = new TodoApi(this.config.get<{ baseUrl: string }>("api").baseUrl);
    this.database = new Database(this.config.get<DatabaseConfig>("database"));
  }
  async initialize(): Promise<void> { await this.database.connect(); }
  async run(): Promise<void> {
    const todos = await this.api.getTodos();
    const newTodo = await this.api.createTodo({ title: "새로운 할 일", completed: false });
  }
}

```

`TodoApp` 이 설정·DB·API를 조립해 돌린다. 각 부품을 클래스로 나누고, `async/await` 로 순서대로 초기화·실행한다. 이 "역할별 클래스로 나누고 조립한다"는 구조가 곧 NestJS의 컨트롤러·서비스·모듈 구조의 축소판이다.

흔한 실수

- `=` 를 쓴다. → `0 = ""`, `50 = "50"` 이 `true` 다. 항상 `===` 를 쓴다.
- `var` 를 쓴다. → 블록을 무시하고 호이스팅 함정에 빠진다. `let` / `const` 만 쓴다.
- `+` 로 숫자를 더했는데 문자열이 이어 붙는다. → 한쪽이 문자열이면 `"25" + 125` 는 `"25125"` 다. 숫자 연산 전 `Number(. )` 로 변환한다.
- `sort` 로 원본이 바뀐다. → `sort` 는 제자리 정렬이라 원본을 변형한다. `.[. arr].sort(. )` 로 복사 후 정렬한다.
- `any` 를 남발한다. → 타입 검사가 꺼져 TS를 쓰는 의미가 없어진다. 정 모를 때만, 그것도 타입 가드로 좁혀 쓴다.
- TS 파일을 `node` 로 돌린다. → `.ts` 는 `node` 가 모른다. `tsx <파일명>` 으로 실행한다.
- `@format()` 이 안 먹는다. → `tsconfig.json` 의 `experimentalDecorators` 가 꺼졌거나 `reflect-metadata` 임포트가 빠졌다.

직접 해보기

1. 배열 고차 함수 예제의 `users` 에서 점수 평균 이상인 사람의 이름만 뽑아 보자. 힌트: `reduce` 로 평균을 구한 뒤 `filter` 로 거르고 `map` 으로 이름만 남긴다.
2. 인터페이스 예제의 `User` 에 `email?: string` 을 추가하고, `Pick` / `Omit` 으로 "이메일만 가진 타입"과 "이메일을 뺀 타입"을 각각 만들어 보자.
3. Node 통합 예제의 `http` 서버에 `/health` 경로를 추가해 `{ status: "ok" }` 를 200으로 응답하게 만들어 보자. 그리고 같은 일을 4장에서 Express로 다시 만들어 두 코드를 비교해 보자.

정리

- JS를 먼저 익히면 타입이 무엇을 해결하는지 보인다. `0 = ""` 의 모호함, `var` 의 함정, `any` 의 위험을 겪어 봐야 TS의 가치가 와닿는다.
- 비동기는 콜백 → Promise → `async/await`로 같은 일을 점점 깔끔하게 적는 것이다. 현업 코드는 `async/await` 가 기본이고, Express·NestJS의 DB 호출이 전부 이 형태다.
- TS는 JS에 인터페이스·제네릭·유틸리티 타입·타입 가드·접근 제어자를 더해, 실행 전에 실수를 잡는다. 데코레이터·싱글턴·역할별 클래스 분리는 곧 NestJS 구조의 바탕이 된다.

다음 장(02 · Node.js 내장 모듈)에서는 이 JavaScript/TypeScript로 `fs`, `path`, `http` 같은 Node의 기본 도구를 다루며, 화면 없는 서버의 토대를 쌓는다.